

Generación de Patrones Sintéticos: Chessboard y Spiral

Brandon Marquez Salazar

Universidad de Guanajuato
October 14, 2025

1 Introducción

Este trabajo presenta la implementación de dos patrones sintéticos: un tablero de ajedrez y un patrón espiral.

2 Métodos

2.1 Generación de Patrones Sintéticos

Se implementaron dos patrones: chessboard y spiral. El primero funcionó correctamente, el segundo presentó problemas en su implementación.

2.1.1 Patrón Chessboard

El patrón chessboard se genera como una matriz M de $n \times m$ píxeles. Cada cuadro tiene dimensiones $w \times h$. El algoritmo sigue un patrón alternante basado en la paridad de las filas.

La implementación en C++ utiliza un enfoque similar con bucles anidados:

```
1 void chessboard(cv::Mat& image, const lovdog::dummystruct& dummy){
2     size_t x,y, i, j, a;
3     image.create(dummy.chess_height * dummy.chess_rows,
4                 dummy.chess_width * dummy.chess_cols, CV_8UC1);
5
6     i=0; while(i<(size_t)image.rows){
7         j=0; while(j<(size_t)image.cols)
8             image.at<uchar>(i,j++)=255;
9         ++i;
10    }
11
12    i=0; while(i<dummy.chess_rows){
13        j=(i&1)*dummy.chess_width;
14        while((int)j<image.cols){
15            a=i*dummy.chess_height;
16            x=0; while(x < dummy.chess_width){
```

Algorithm 1 Generación de Chessboard

```
1: procedure CHESSBOARD( $n, m, w, h$ )
2:    $M \leftarrow 255$  ▷ Inicializar blanco
3:   for  $i = 0$  to  $n - 1$  do
4:      $j_{start} \leftarrow (i \bmod 2) \cdot w$ 
5:     for  $j = j_{start}$  to  $m - 1$  step  $2w$  do
6:       for  $x = 0$  to  $w - 1$  do
7:         for  $y = 0$  to  $h - 1$  do
8:            $M[i \cdot h + y, j + x] \leftarrow 0$ 
9:         end for
10:      end for
11:    end for
12:  end for
13:  return  $M$ 
14: end procedure
```

```
17     y=0; while(y < dummy.chess_height){
18         image.at<uchar>(a+(y++),j+x) = 0;
19     }
20     ++x;
21 }
22 j+=2*dummy.chess_width;
23 }
24 ++i;
25 }
26 }
```

Listing 1: Implementación Chessboard

2.1.2 Patrón Spiral: Planificación del Algoritmo

Para el patrón spiral, se planeó un algoritmo que siguiera un rastro similar a un barbechado, evitando cruzar las restricciones de los márgenes. La idea central era usar un índice circular sobre un vector con los cuatro movimientos posibles para lograr un avance fluido.

La matriz $M \in R^{n \times m}$ no es necesariamente cuadrada. Sea g el grosor de la línea que formará la espiral y también el espacio entre sus paredes.

Los márgenes se calculan como:

- Margen superior: $m_{sp} = \lfloor m \bmod g \rfloor$
- Margen inferior: $m_{in} = m \bmod g - m_{sp}$
- Margen izquierdo: $m_{iz} = \lfloor n \bmod g \rfloor$
- Margen derecho: $m_{dr} = n \bmod g - m_{iz}$

El punto de inicio es $P_i = (m_{iz}, m_{sp})$. El cambio de dirección siempre es a la derecha, usando un índice circular sobre el vector de movimientos $\vec{d} = \{(1, 0), (0, 1), (-1, 0), (0, -1)\}$.

Los rangos para cada caso se definen como:

Algorithm 2 Planificación del Algoritmo Spiral

```

1: procedure SPIRAL( $n, m, g$ )
2:    $M \leftarrow 255$  ▷ Fondo blanco
3:   Calcular  $m_{sp}, m_{in}, m_{iz}, m_{dr}$ 
4:    $P \leftarrow (m_{iz}, m_{sp})$  ▷ Punto inicial
5:    $\vec{d} \leftarrow \{(1, 0), (0, 1), (-1, 0), (0, -1)\}$ 
6:    $idx \leftarrow 0$  ▷ Índice direccional
7:    $a, b, c, d \leftarrow 0$  ▷ Contadores de ajuste progresivo
8:   while espacio disponible do
9:     if dirección horizontal then
10:       $x \in [a + m_{iz}, m_{dr} - b]$  ▷ Rango horizontal
11:       $y \in [c + m_{sp}, c + m_{sp} + g]$  ▷ Grosor vertical
12:      Dibujar línea horizontal
13:       $c \leftarrow c + g$  ▷ Ajustar para siguiente segmento
14:     else
15:       $x \in [a + m_{iz}, a + m_{iz} + g]$  ▷ Grosor horizontal
16:       $y \in [c + m_{sp}, m_{in} - d]$  ▷ Rango vertical
17:      Dibujar línea vertical
18:       $a \leftarrow a + g$  ▷ Ajustar para siguiente segmento
19:     end if
20:      $idx \leftarrow (idx + 1) \bmod 4$  ▷ Cambio cíclico de dirección
21:     Actualizar  $b, d$  según corresponda
22:   end while
23:   return  $M$ 
24: end procedure

```

La idea del barbechado consiste en que cada vez que se completa un segmento horizontal o vertical, los contadores a, b, c, d se incrementan en g , haciendo que el área disponible para el siguiente segmento se reduzca. Esto crea el efecto de espiral que se va cerrando progresivamente.

El punto de inflexión para cambiar la dirección ocurre cuando el algoritmo encuentra el límite de sus posibles movimientos, detectado por las condiciones de los rangos. La transición entre direcciones debe ser suave, manteniendo la continuidad del patrón.

3 Resultados

3.1 Patrón Chessboard

El algoritmo de chessboard generó el patrón esperado. La figura muestra el resultado con parámetros $w = 40$, $h = 20$, 8×8 cuadros.

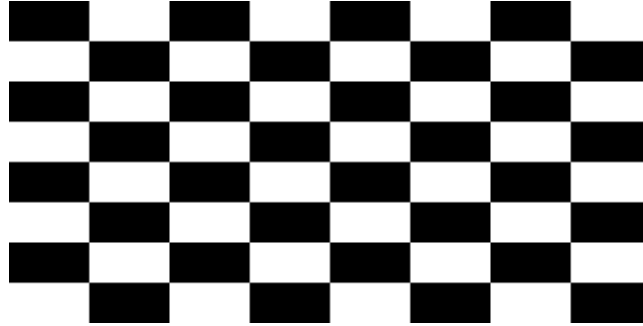


Figure 1: Patrón chessboard generado por el algoritmo.

El patrón muestra alternancia correcta entre cuadros blancos y negros. Los bordes son nítidos y las dimensiones se mantienen consistentes en toda la imagen.

3.2 Patrón Spiral

La implementación del patrón spiral no funcionó correctamente. Aunque la planificación del algoritmo era sólida en teoría, la implementación práctica presentó problemas con la sincronización de los contadores a, b, c, d y la detección precisa de los puntos de inflexión.

El algoritmo generaba segmentos desconectados o patrones que no correspondían a una espiral cerrada. La transición entre direcciones no era suave y en ocasiones se producían solapamientos o huecos en el patrón.

4 Conclusiones

El patrón chessboard se implementó correctamente usando un enfoque basado en paridad de filas. Para el patrón spiral, la planificación teórica con el sistema de barbechado y ajuste progresivo de contadores era adecuada, pero la implementación requirió un manejo más cuidadoso de las transiciones entre segmentos y la sincronización de los límites.